

App Development Project Report

Trekking Management Application

1. Student Details

Name: Arya Mulay

IITM Roll No.: 23f3002123

IITM email id: 23f3002123@ds.study.iitm.ac.in

IITM github id: <https://github.com/23f3002123>

Personal github id: <https://github.com/aryasm13>

About Me: I am a dual degree student at IIT Madras BS Degree program and VIT Vellore pursuing IT with a deep interest in data engineering, Deep Learning Algorithms and software engineering and secure web application development.

2. Project Details

Project Title: Trekking Management Application

Problem Statement: To design and build a role-based capstone web application allowing Administrators, Trek Staff, and Trekkers to interact based on validation-enforced permissions, coordinate trek bookings, prevent slots overbooking, track ongoing states, and query booking logs.

Approach: The application was built as a modular Flask application structure using blueprints. All database records (users, guide profiles, treks, and bookings) are programmatically managed in SQLite. Data updates and actions are structured to run safely locally with robust session checks.

3. AI/LLM Declaration

I used ChatGPT to assist in writing SQLAlchemy relationship bindings, structuring HTML form templates, and formatting layout CSS variables.

I also have generated the background image (which is rendered on all pages) via Google's Gemini and used AntigraVity to debug the codebase for any errors I might have missed.

The extent of AI/LLM usage is approximately **15-20%**. All validation checks, controller security filters, database schema constraints, and business logic implementation were completed manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework managing routing and requests
SQLAlchemy	Object Relational Mapper managing SQLite interactions
Jinja2	Template engine for rendering dynamic HTML views
Bootstrap 5	Responsive grid design and default layout rules
SQLite	Embedded SQL database engine
Chart.js	Generates canvas-based dashboard statistics
Flask-Login	Manages cookie sessions and login decorators
Werkzeug	Enforces cryptographic password hashing and verification

5. Database Schema / ER Diagram

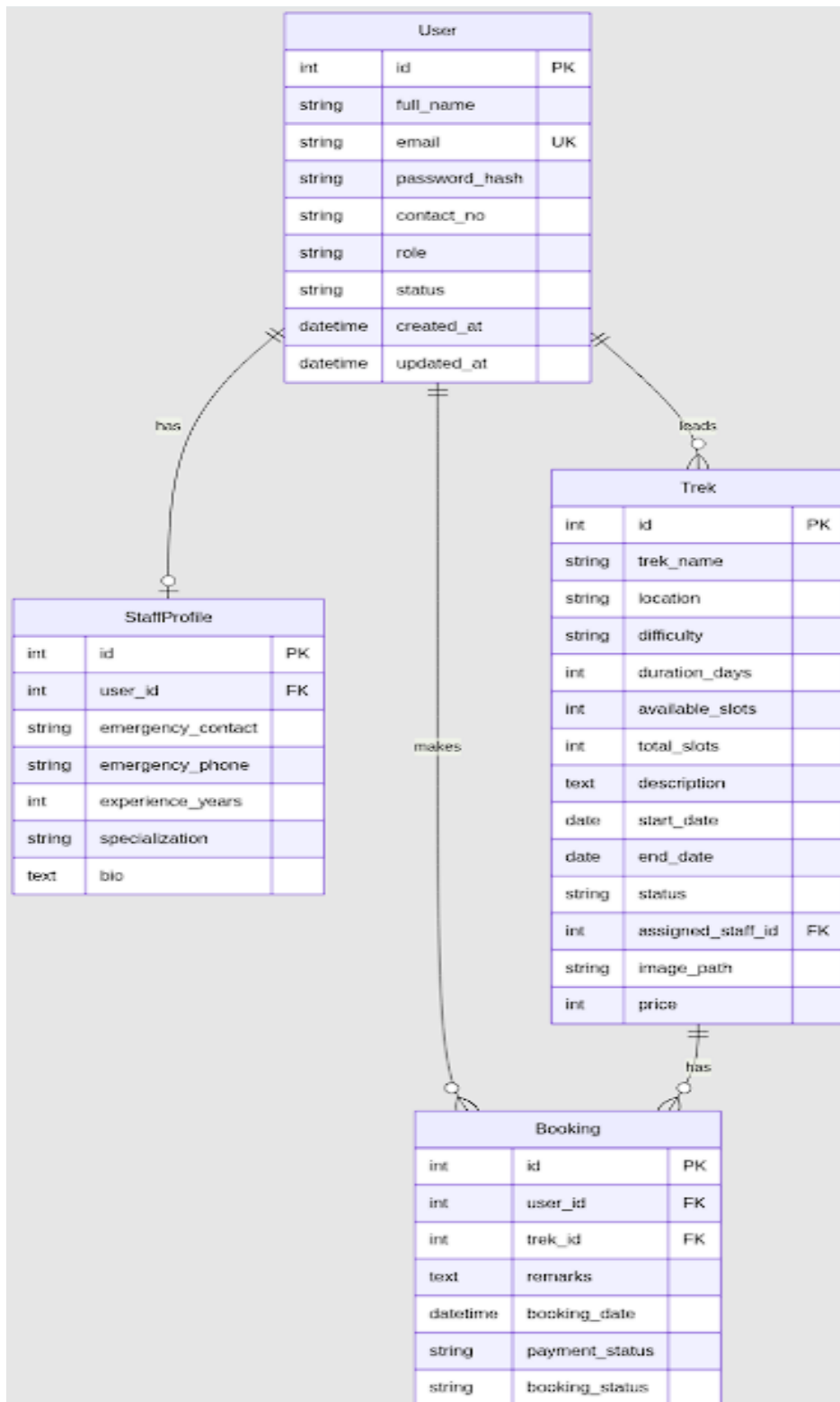
The database layout comprises four normalized tables. Unique indices, foreign keys, and SQL constraints preserve data hygiene.

- **User:** Stores user profiles, emails, role ('admin'/'staff'/'trekker'), and status ('pending'/'approved'/'blacklisted').
- **StaffProfile:** Contains guide attributes (specialization, bio, experience, emergency contacts) pointing back to User id.
- **Trek:** Details trekking events (name, duration, start/end dates, image paths, slots, prices, assigned staff guide).
- **Booking:** Tracks registrations with payment_status ('Pending'/'Paid'/'Refunded') and booking_status ('Booked'/'Cancelled'/'Completed').

Relationships:

- One-to-One : User => StaffProfile (via user_id ForeignKey)
- One-to-Many: User => Booking (a user can book multiple treks)
- One-to-Many: User (Staff) => Trek (a guide leads multiple assigned treks)
- One-to-Many: Trek => Booking (a trek has multiple participant bookings)

ER Diagram:



6. Application Endpoints (UI & API Routes)

Endpoint URL	Method	Functional Description
/login	GET, POST	Renders login page and logs in users, staff, or admins
/register	GET, POST	Handles self-registration for trekkers and guides
/logout	POST	Clears browser cookie session and logs out user
/admin/dashboard	GET	Shows admin metrics summaries and recent bookings
/admin/treks	GET	Allows viewing and filtering of all created treks
/admin/trek/new	GET, POST	Allows admin to build and setup a new trek event
/admin/trek/<id>/edit	GET, POST	Edits trek location, slots, pricing, and assigned staff
/admin/staff	GET	Lists all guides and supports status updates
/admin/users	GET	Lists all registered trekkers with blacklist options
/admin/search	GET	Performs database-wide queries on treks, staff, and trekkers
/staff/dashboard	GET	Displays assigned guide treks and booking statistics
/staff/profile	GET, POST	Allows guide to fill experience years, specialization, and bio
/staff/trek/<id>	GET	Displays trek detail and participants table list
/staff/trek/<id>/update	POST	Allows guides to alter available slots and status
/user/dashboard	GET	Displays trekker stats, open treks table, and booking graphs
/user/profile	GET, POST	Enables trekker details updates (names, phone)
/user/treks	GET	Browse and filter approved treks by location/difficulty
/user/trek/<id>	GET	Shows detailed information, slots left, and price
/user/trek/<id>/book	POST	Enables users to reserve a spot (checks available slots)
/user/bookings	GET	Lists booked treks with checkout and cancellation options
/user/booking/<id>/pay	GET, POST	Simple checkout mock payment page
/user/booking/<id>/cancel	POST	Cancels booking and increases available slots

7. Architecture and Features

Architecture: This application uses blueprints to modularize routing. The logic resides inside routes/controllers while views are rendered via Jinja2 layouts in templates/. DB states are accessed safely using Python SQLAlchemy ORM objects.

Core & Extra Features: Session protection, overbooking prevention via SQLite db checks, staff approval workflows, ongoing trek tracking dashboards, custom CSS background layout, Chart.js integrations for basic charts on dashboards, and a mock billing checkout portal.

8. Video Presentation and G-drive Folder Links:

[Video Presentation Link](#)

[Google Drive Folder Link](#)